

Ejercicio 3: Modelo Vectorial y TF-IDF

Objetivo de la práctica

- Comprender el modelo vectorial como base para representar documentos y consultas.
- Calcular la matriz TF-IDF para el corpus data/01_corpus_turismo_500.txt
- Calcular la matriz TF-IDF para el corpus Gutenberg 1000

Paso 1: Calcular la matriz TF-IDF para el corpus data/01_corpus_turismo_500.txt

```
!pip install numpy==1.26.4 scipy==1.13.1 scikit-learn==1.5.2
```

Collecting numpy==1.26.4

Downloading numpy-1.26.4-cp312-cp312-win_amd64.whl.metadata (61 kB)

Requirement already satisfied: scipy==1.13.1 in c:\users\ladol\anaconda3\lib\site-packages (

Collecting scikit-learn==1.5.2

Downloading scikit_learn-1.5.2-cp312-cp312-win_amd64.whl.metadata (13 kB)

Requirement already satisfied: joblib>=1.2.0 in c:\users\ladol\anaconda3\lib\site-packages (

Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\ladol\anaconda3\lib\site-pac

Downloading numpy-1.26.4-cp312-cp312-win_amd64.whl (15.5 MB)

```
----- 0.0/15.5 MB ? eta -:--:--
----- 0.3/15.5 MB ? eta -:--:--
-- ----- 0.8/15.5 MB 2.0 MB/s eta 0:00:08
--- ----- 1.3/15.5 MB 2.3 MB/s eta 0:00:07
---- ----- 1.8/15.5 MB 2.4 MB/s eta 0:00:06
----- 2.4/15.5 MB 2.5 MB/s eta 0:00:06
----- 2.9/15.5 MB 2.6 MB/s eta 0:00:05
----- 3.4/15.5 MB 2.6 MB/s eta 0:00:05
----- 3.9/15.5 MB 2.7 MB/s eta 0:00:05
----- 4.5/15.5 MB 2.7 MB/s eta 0:00:05
----- 4.7/15.5 MB 2.4 MB/s eta 0:00:05
----- 5.0/15.5 MB 2.4 MB/s eta 0:00:05
```

```

----- 5.5/15.5 MB 2.4 MB/s eta 0:00:05
----- 6.3/15.5 MB 2.4 MB/s eta 0:00:04
----- 6.8/15.5 MB 2.5 MB/s eta 0:00:04
----- 7.3/15.5 MB 2.5 MB/s eta 0:00:04
----- 7.6/15.5 MB 2.4 MB/s eta 0:00:04
----- 8.1/15.5 MB 2.4 MB/s eta 0:00:04
----- 8.7/15.5 MB 2.4 MB/s eta 0:00:03
----- 8.9/15.5 MB 2.4 MB/s eta 0:00:03
----- 9.4/15.5 MB 2.4 MB/s eta 0:00:03
----- 10.2/15.5 MB 2.4 MB/s eta 0:00:03
----- 10.7/15.5 MB 2.5 MB/s eta 0:00:02
----- 11.3/15.5 MB 2.5 MB/s eta 0:00:02
----- 11.8/15.5 MB 2.5 MB/s eta 0:00:02
----- 12.3/15.5 MB 2.5 MB/s eta 0:00:02
----- 12.6/15.5 MB 2.5 MB/s eta 0:00:02
----- 12.6/15.5 MB 2.5 MB/s eta 0:00:02
----- 13.1/15.5 MB 2.3 MB/s eta 0:00:02
----- 13.6/15.5 MB 2.3 MB/s eta 0:00:01
----- 13.9/15.5 MB 2.3 MB/s eta 0:00:01
----- 13.9/15.5 MB 2.3 MB/s eta 0:00:01
----- 14.2/15.5 MB 2.2 MB/s eta 0:00:01
----- 14.4/15.5 MB 2.2 MB/s eta 0:00:01
----- 14.4/15.5 MB 2.2 MB/s eta 0:00:01
----- 14.7/15.5 MB 2.1 MB/s eta 0:00:01
----- 14.9/15.5 MB 2.1 MB/s eta 0:00:01
----- 14.9/15.5 MB 2.1 MB/s eta 0:00:01
----- 15.5/15.5 MB 2.0 MB/s eta 0:00:01
----- 15.5/15.5 MB 2.0 MB/s eta 0:00:00
Downloading scikit_learn-1.5.2-cp312-cp312-win_amd64.whl (11.0 MB)
----- 0.0/11.0 MB ? eta -:--:--
----- 0.3/11.0 MB ? eta -:--:--
-- ----- 0.8/11.0 MB 2.1 MB/s eta 0:00:05
----- 1.3/11.0 MB 2.2 MB/s eta 0:00:05
----- 1.6/11.0 MB 2.2 MB/s eta 0:00:05
----- 2.1/11.0 MB 2.3 MB/s eta 0:00:04
----- 2.4/11.0 MB 2.2 MB/s eta 0:00:04
----- 2.9/11.0 MB 2.2 MB/s eta 0:00:04
----- 3.1/11.0 MB 2.1 MB/s eta 0:00:04
----- 3.4/11.0 MB 2.0 MB/s eta 0:00:04
----- 3.9/11.0 MB 2.0 MB/s eta 0:00:04
----- 4.5/11.0 MB 2.0 MB/s eta 0:00:04
----- 4.7/11.0 MB 2.0 MB/s eta 0:00:04
----- 5.2/11.0 MB 2.0 MB/s eta 0:00:03

```

```

----- 5.8/11.0 MB 2.0 MB/s eta 0:00:03
----- 6.3/11.0 MB 2.1 MB/s eta 0:00:03
----- 6.8/11.0 MB 2.1 MB/s eta 0:00:02
----- 7.1/11.0 MB 2.1 MB/s eta 0:00:02
----- 7.6/11.0 MB 2.1 MB/s eta 0:00:02
----- 8.1/11.0 MB 2.1 MB/s eta 0:00:02
----- 8.1/11.0 MB 2.1 MB/s eta 0:00:02
----- 8.4/11.0 MB 2.0 MB/s eta 0:00:02
----- 8.7/11.0 MB 1.9 MB/s eta 0:00:02
----- 8.9/11.0 MB 1.9 MB/s eta 0:00:02
----- 9.2/11.0 MB 1.9 MB/s eta 0:00:01
----- 9.7/11.0 MB 1.9 MB/s eta 0:00:01
----- 10.0/11.0 MB 1.9 MB/s eta 0:00:01
----- 10.5/11.0 MB 2.0 MB/s eta 0:00:01
----- 10.7/11.0 MB 1.9 MB/s eta 0:00:01
----- 11.0/11.0 MB 1.9 MB/s eta 0:00:00

```

Installing collected packages: numpy, scikit-learn

Attempting uninstall: numpy

Found existing installation: numpy 2.3.2

Uninstalling numpy-2.3.2:

Successfully uninstalled numpy-2.3.2

Attempting uninstall: scikit-learn

Found existing installation: scikit-learn 1.5.1

Uninstalling scikit-learn-1.5.1:

Successfully uninstalled scikit-learn-1.5.1

Successfully installed numpy-1.26.4 scikit-learn-1.5.2

WARNING: Ignoring invalid distribution ~atplotlib (C:\Users\ladol\anaconda3\Lib\site-packages)

WARNING: Ignoring invalid distribution ~atplotlib (C:\Users\ladol\anaconda3\Lib\site-packages)

WARNING: Failed to remove contents in a temporary directory 'C:\Users\ladol\anaconda3\Lib\site-packages\~atplotlib'.
You can safely remove it manually.

WARNING: Failed to remove contents in a temporary directory 'C:\Users\ladol\anaconda3\Lib\site-packages\~atplotlib'.
You can safely remove it manually.

WARNING: Ignoring invalid distribution ~atplotlib (C:\Users\ladol\anaconda3\Lib\site-packages)

ERROR: pip's dependency resolver does not currently take into account all the packages that are required by packages you have installed. For example, prophet 1.1.7 requires matplotlib>=2.0.0, which is not installed.

seaborn 0.13.2 requires matplotlib!=3.6.1,>=3.4, which is not installed.

streamlit 1.37.1 requires packaging<25,>=20, but you have packaging 25.0 which is incompatible.

streamlit 1.37.1 requires pillow<11,>=7.1.0, but you have pillow 11.3.0 which is incompatible.

```
# TF-IDF (Term Frequency - Inverse Document Frequency)
```

```

# Idea central: una palabra es importante en un documento si:
#   - Aparece MUCHO en ese documento (TF alto)
#   - Pero NO aparece en todos los demás (IDF alto)

# Fórmulas:
#   TF(t, d)      = frecuencia del término t en el documento d
#   IDF(t)        = log(N / df(t)) donde N = total docs, df(t) = docs con t
#   TF-IDF(t, d) = TF(t, d) × IDF(t)

# El resultado es una MATRIZ: cada FILA = un documento representado como
# un vector numérico en el espacio del vocabulario.

import os
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer

# Cargar corpus de turismo
# El archivo tiene 500 líneas; cada línea es un "documento" independiente.
ruta_turismo = os.path.join(os.getcwd(), '01_corpus_turismo_500.txt')

with open(ruta_turismo, 'r', encoding='utf-8') as f:
    corpus_turismo = f.read().splitlines() # lista de 500 strings

print(f'Número de documentos: {len(corpus_turismo)}')
print('Ejemplo de documento:')
print(corpus_turismo[0])

```

Número de documentos: 500

Ejemplo de documento:

Otavalo es conocido por su mercado indígena y su artesanía Perfecto para rafting.

```

# Crear y ajustar el vectorizador TF-IDF

# TfidfVectorizer de sklearn hace todo en un solo paso:
#   1. Tokeniza cada documento (lo separa en palabras)
#   2. Construye el vocabulario (cada palabra única = una columna)
#   3. Calcula TF-IDF para cada par (documento, término)

# Parámetros clave:
#   - max_df=0.95 : ignora términos que aparecen en más del 95% de los docs
#                   (son tan comunes que no distinguen nada, como stopwords)

```

```

# - min_df=2      : ignora términos que aparecen en menos de 2 documentos
#                  (tan raros que tampoco ayudan a discriminar)

vectorizador_turismo = TfidfVectorizer(max_df=0.95, min_df=2)

# fit_transform hace dos cosas a la vez:
#   fit()          -> aprende el vocabulario y los valores IDF del corpus
#   transform()-> convierte cada documento a su vector TF-IDF
matriz_turismo = vectorizador_turismo.fit_transform(corpus_turismo)

# La matriz es 'sparse' (dispersa): solo almacena los valores != 0
# porque la mayoría de palabras NO aparecen en cada documento.
print(f'Forma de la matriz TF-IDF: {matriz_turismo.shape}')
print(f'  → {matriz_turismo.shape[0]} documentos × {matriz_turismo.shape[1]} términos del vo

# Densidad: porcentaje de celdas que tienen valor distinto de cero
densidad = matriz_turismo.nnz / (matriz_turismo.shape[0] * matriz_turismo.shape[1])
print(f'Densidad (valores != 0): {densidad:.2%}')

```

Forma de la matriz TF-IDF: (500, 116)
 → 500 documentos × 116 términos del vocabulario
 Densidad (valores != 0): 9.90%

```

# Explorar la matriz como DataFrame
# Convertimos a DataFrame solo para visualización.
# En producción se trabaja con la matriz sparse (menos memoria).

vocabulario_turismo = vectorizador_turismo.get_feature_names_out()

df_turismo = pd.DataFrame(
    matriz_turismo.toarray(),      # .toarray() convierte sparse → numpy
    columns=vocabulario_turismo
)

print('Primeras 5 filas y 10 columnas de la matriz TF-IDF:')
df_turismo.iloc[:5, :10]

```

Primeras 5 filas y 10 columnas de la matriz TF-IDF:

	2000	agua	amazonía	arquitectura	artesanía	atrae	atraen	auténtico	aventura	aves
0	0.0	0.0	0.0	0.0	0.341664	0.000000	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	0.0	0.000000	0.000000	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.000000	0.352958	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.000000	0.000000	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.000000	0.000000	0.0	0.0	0.0	0.0

```
# ¿Qué palabras tienen mayor TF-IDF promedio?
# Las palabras con mayor promedio son las más "discriminativas".

tfidf_promedio = df_turismo.mean(axis=0).sort_values(ascending=False)

print('Top 15 términos por TF-IDF promedio:')
print(tfidf_promedio.head(15))
```

Paso 2: Construir el corpus Gutenberg 1000

El corpus Gutenberg 1000 es un corpus compuesto por 1000 libros de Gutenberg Project

```
# Construir el corpus Gutenberg
# En Recuperación de Información, un 'corpus' es la colección de documentos
# sobre la que vamos a hacer búsquedas. Aquí cada LIBRO es un documento.
# Usamos los libros de la carpeta '1000libros/' descargados previamente.

ruta_libros = os.path.join(os.getcwd(), '1000libros') # carpeta con los libros descargados
archivos = sorted(os.listdir(ruta_libros)) # lista de nombres de archivo

print(f'Libros disponibles: {len(archivos)}')

# Cargamos cada libro como un elemento de la lista.
# También guardamos los nombres para identificar resultados.
corpus_gutenberg = []
nombres_gutenberg = []

for archivo in archivos:
    ruta_archivo = os.path.join(ruta_libros, archivo)
    try:
        with open(ruta_archivo, 'r', encoding='utf-8') as f:
            texto = f.read()
            corpus_gutenberg.append(texto)
```

```

        nombres_gutenberg.append(archivo)
    except Exception as e:
        print(f' Error al leer {archivo}: {e}')

print(f'Libros cargados correctamente: {len(corpus_gutenberg)}')
print(f'Ejemplo: "{nombres_gutenberg[0]}" - {len(corpus_gutenberg[0])} caracteres')

```

Libros disponibles: 997

Libros cargados correctamente: 997

Ejemplo: "A Case in Camera.txt" - 477114 caracteres

Paso 3: Calcular la matriz TF-IDF para el corpus Gutenberg 1000

```

# TF-IDF sobre el corpus de libros
# Diferencias clave respecto al corpus de turismo:
#   - Cada documento es un LIBRO ENTERO (cientos de miles de palabras)
#   - El vocabulario será MUCHO mayor
#   - La matriz tendrá pocas filas pero muchas columnas
# Parámetros ajustados para libros grandes:
#   - max_features=50000 : limita el vocabulario a las 50,000 palabras
#     más frecuentes para no agotar la memoria
#   - sublinear_tf=True : aplica log(1 + TF) en lugar de TF puro;
#     evita que documentos muy largos dominen solo por tener más palabras

import time

vectorizador_gutenberg = TfidfVectorizer(
    max_features=50000, # limitar vocabulario para no agotar memoria
    sublinear_tf=True, # log(1+TF) - más justo para docs de distinto tamaño
    max_df=0.95,       # descarta términos en más del 95% de los libros
    min_df=2           # descarta términos que aparecen en un solo libro
)

# fit_transform: aprende vocabulario de todos los libros y construye la matriz
t0 = time.time()
matriz_gutenberg = vectorizador_gutenberg.fit_transform(corpus_gutenberg)
t1 = time.time()

print(f'Tiempo de vectorización: {t1 - t0:.2f} s')
print(f'Forma de la matriz: {matriz_gutenberg.shape}')

```

```
print(f' → {matriz_gutenberg.shape[0]} libros × {matriz_gutenberg.shape[1]} términos')
print(f'Valores no cero: {matriz_gutenberg.nnz:,} ({matriz_gutenberg.nnz / (matriz_gutenberg
```

Tiempo de vectorización: 93.50 s
 Forma de la matriz: (997, 50000)
 → 997 libros × 50000 términos
 Valores no cero: 8,476,716 (17.00% de densidad)

```
# Términos más representativos de un libro específico
# Un valor alto de TF-IDF significa que la palabra es MUY frecuente
# en ese libro pero RARA en los demás → es un "término característico".

import numpy as np

idx_libro = 0 # cambia este índice para explorar otros libros

vocabulario_gutenberg = vectorizador_gutenberg.get_feature_names_out()
vector_libro = matriz_gutenberg[idx_libro].toarray().flatten()

# Ordenamos los índices de mayor a menor TF-IDF
top_indices = np.argsort(vector_libro)[::-1][:10]

print(f'Términos más representativos de "{nombres_gutenberg[idx_libro]}":')
for i in top_indices:
    print(f' {vocabulario_gutenberg[i]:<25} TF-IDF = {vector_libro[i]:.4f}')
```

Términos más representativos de "A Case in Camera.txt":

monty	TF-IDF = 0.1267
esdaile	TF-IDF = 0.1260
mollie	TF-IDF = 0.1002
chummy	TF-IDF = 0.0947
rooke	TF-IDF = 0.0877
audrey	TF-IDF = 0.0799
westbury	TF-IDF = 0.0754
hubbard	TF-IDF = 0.0716
bobby	TF-IDF = 0.0586
cunningham	TF-IDF = 0.0577

Paso 4: Programar una función buscar() para el corpus Gutenberg 1000

```
# Búsqueda por similitud coseno con TF-IDF
# ¿Por qué similitud coseno?
#   Mide el ÁNGULO entre dos vectores, no la distancia euclidiana.
#   Valor entre 0 (sin relación) y 1 (idénticos en contenido).
#   Es robusta ante diferencias de longitud: un libro largo y uno corto
#   sobre el mismo tema tendrán coseno alto aunque sus vectores tengan
#   magnitudes muy distintas.

# Proceso de la búsqueda:
#   1. Transformar la consulta en un vector TF-IDF (usando el vectorizador
#       ya entrenado - NO se re-entrena con la consulta)
#   2. Calcular similitud coseno entre vector-consulta y CADA fila
#   3. Ordenar de mayor a menor similitud
#   4. Devolver los top-k más similares

from sklearn.metrics.pairwise import cosine_similarity

def buscar(query: str, top_k: int = 5) -> list[tuple[str, float]]:
    #Busca los libros más relevantes para una consulta usando TF-IDF + coseno.
    #Args:
        #query : texto de la consulta (ej. "amor y muerte")
        #top_k : cuántos resultados devolver (por defecto 5)
    #Returns:
        #Lista de tuplas (nombre_libro, score) ordenadas de mayor a menor.
    # transform() (sin fit) convierte la consulta al MISMO espacio vectorial.
    # Las palabras que no están en el vocabulario se ignoran.
    vector_query = vectorizador_gutenberg.transform([query]) # shape (1, vocabulario)

    # cosine_similarity devuelve una matriz (1 × N_libros)
    scores = cosine_similarity(vector_query, matriz_gutenberg).flatten()

    # argsort ordena de menor a mayor → invertimos con [::-1]
    indices_ordenados = np.argsort(scores)[::-1][:top_k]

    # Construimos la lista de resultados
    resultados = [(nombres_gutenberg[i], round(scores[i], 4)) for i in indices_ordenados]
    return resultados
```

```
# Probar la función buscar() con varias consultas

consultas = [
    "amor y muerte",
    "historia de América",
    "aventura y viaje",
    "poesía lírica",
]

for consulta in consultas:
    print(f'\nConsulta: "{consulta}"')
    print('-' * 50)
    for libro, score in buscar(consulta, top_k=3):
        print(f' [{score:.4f}] {libro}')
```

Consulta: "amor y muerte"

```
-----
[0.0397] Don Quijote.txt
[0.0375] El crimen y el castigo.txt
[0.0317] Diccionario Ingles-Español-Tagalog Con partes de la oracion y pronunciacion figur
```

Consulta: "historia de América"

```
-----
[0.0634] The Spanish American Reader.txt
[0.0480] The Audiencia in the Spanish Colonies As illustrated by the Audiencia of Manila (
[0.0349] History of Central America, Volume 3, 1801-1887 The Works of Hubert Howe Bancroft
```

Consulta: "aventura y viaje"

```
-----
[0.0534] Don Quijote.txt
[0.0382] El crimen y el castigo.txt
[0.0364] Diccionario Ingles-Español-Tagalog Con partes de la oracion y pronunciacion figur
```

Consulta: "poesía lírica"

```
-----
[0.0000] Æsthetic as science of expression and general linguistic.txt
[0.0000] Life Blood.txt
[0.0000] London and the Kingdom - Volume 2 A History Derived Mainly from the Archives at G
```

```

# Búsqueda interactiva
# Modifica 'mi_consulta' y vuelve a ejecutar para buscar en el corpus.

mi_consulta = "Don Quijote caballero" # ← cambia aquí tu consulta

resultados = buscar(mi_consulta, top_k=5)

print(f'Resultados para: "{mi_consulta}"')
print('=' * 50)
for rank, (libro, score) in enumerate(resultados, start=1):
    print(f'{rank}. [{score:.4f}] {libro}')

```

Resultados para: "Don Quijote caballero"

=====

1. [0.0755] Don Quijote.txt
2. [0.0351] The Philippines a Century Hence.txt
3. [0.0275] The Treasure of Pearls A Romance of Adventures in California.txt
4. [0.0253] The Memoirs of the Conquistador Bernal Diaz del Castillo, Vol 1 (of 2) Written by
5. [0.0178] Minute Mysteries [Detectograms].txt